



1. Cache Memory — Direct Mapping

QUANTITY	FORMULA
Number of bits in Block Offset	$b = \log_2(\text{Block Size})$
Number of bits in Line/Index	$L = \log_2(\text{Number of Lines in Cache})$
Number of bits in Tag	$T = \text{Total Address Bits} - L - b$
Line Number for a given address	$\text{Line} = (\text{Block Address}) \bmod (\text{Number of Lines})$



COMMON PATTERN:

Given main memory size, cache size, block size → find tag/index/offset bits, OR given an address → find which cache line it maps to and whether two addresses conflict.

SHORTCUT:

Always compute b first (block size is usually a power of 2 given directly), then $L = \log_2(\text{cache size/block size})$, then $T = \text{remainder}$. Two addresses conflict (map to same line) in direct mapping iff their LINE bits are equal (tag may differ).

PYQ STYLE:

"32-bit address, cache size 64 KB, block size 32 B, direct mapped. Find number of tag bits." → $\text{Lines} = 64\text{KB}/32\text{B} = 2048 = 2^{11} \rightarrow L=11, b=5, T=32-11-5=16$.

2. Set Associative Mapping

QUANTITY	FORMULA
Number of Sets	Sets = Cache Size / (Block Size × Ways)
Set Index bits	$S = \log_2(\text{Number of Sets})$
Tag bits	$T = \text{Total Address Bits} - S - b$

COMMON PATTERN:

"k-way set associative cache" — find number of sets, comparators needed, or tag bits, given k.

SHORTCUT:

Direct Mapping = 1-way Set Associative. Fully Associative = (Cache Size/Block Size)-way Set Associative. Number of comparators needed = number of ways (k), each comparing the Tag field in parallel.

PYQ STYLE:

"Cache 16 KB, block 16 B, 4-way set associative." → Sets = $16K/(16 \times 4) = 256 = 2^8 \rightarrow S = 8$ bits.

3. Fully Associative Mapping

QUANTITY	FORMULA
Index bits	0 (no index field – any block can go anywhere)
Tag bits	$T = \text{Total Address Bits} - b$
Comparators needed	= Number of cache lines (all compared in parallel)

COMMON PATTERN:

Compare tag-bit count across direct / set-associative / fully-associative for the same cache — fully associative always needs the MOST tag bits and MOST hardware (comparators).

SHORTCUT:

As associativity ↑, conflict misses ↓ but hardware cost (comparators) ↑. Tag bits increase as ways increase (since index bits shrink).

4. Hit Ratio, Miss Ratio & Average Memory Access Time (AMAT)

QUANTITY	FORMULA
Hit Ratio (h)	$h = \text{Hits} / \text{Total References}$
Miss Ratio	$(1 - h)$
AMAT (single level)	$AMAT = h \times T_{\text{cache}} + (1 - h) \times T_{\text{memory}}$
AMAT (with miss penalty form)	$AMAT = T_{\text{cache}} + (1 - h) \times \text{Miss Penalty}$
Multi-level cache AMAT	$AMAT = T_1 + (1-h_1) \times [T_2 + (1-h_2) \times T_{\text{mem}}]$

COMMON PATTERN:

Given cache access time, memory access time, and hit ratio → compute AMAT. Or given target AMAT → find required hit ratio (reverse problem, very common).

SHORTCUT:

When the question gives "effective access time," it almost always means AMAT — just plug into the formula. For 2-level cache, treat L2 exactly like "memory" from L1's perspective, then nest.

PYQ STYLE:

Cache access = 10 ns, memory access = 100 ns, hit ratio = 0.9. $AMAT = 0.9(10) + 0.1(100) = 9 + 10 = 19 \text{ ns}$.

5. TLB & Effective Access Time (Paging)

QUANTITY	FORMULA
EAT with TLB (no page fault)	$EAT = h \times (T_{lb} + T_{mem}) + (1-h) \times (T_{lb} + 2 \times T_{mem})$
EAT with page fault possible	$EAT = h \times (T_{lb} + T_{mem}) + (1-h) \times [T_{lb} + \text{Page Fault Service Time}]$
EAT without TLB	$EAT = 2 \times T_{mem}$ (1 for page table, 1 for actual data)

COMMON PATTERN:

"TLB hit ratio = $h\%$, TLB access = x ns, memory access = y ns. Find effective access time."
Sometimes extended with multi-level page tables (add one T_{mem} per level).

SHORTCUT:

Count memory accesses needed in each case carefully — TLB hit = 1 memory access (data);
TLB miss = 1 access for page table + 1 for data = 2. Multiply each path's time by its probability and add.

PYQ STYLE:

TLB access = 20 ns, memory = 100 ns, hit ratio = 80%. $EAT = 0.8(20+100) + 0.2(20+200) = 96+44 = 140$ ns.

6. Pipelining — Speedup, Efficiency, Throughput

QUANTITY	FORMULA
Time for n instructions (k-stage pipeline)	$T = (k + n - 1) \times T_p$
Speedup	$S = (n \times k \times T_p) / [(k + n - 1) \times T_p] = nk / (k+n-1)$
Max Speedup ($n \rightarrow \infty$)	$S(\text{max}) = k$ (number of stages)
Efficiency	$E = S / k$
Throughput	Throughput = $n / [(k+n-1) \times T_p]$ instructions/sec
With Stalls (hazards)	$T = (k + n - 1 + \text{stalls}) \times T_p$

Pipeline Timing Diagram (k=4 stages, n=4 instructions)



COMMON PATTERN:

Given clock period, number of stages, number of instructions → compute total execution time / speedup. Sometimes hazard stalls added per instruction.

SHORTCUT:

Memorize " $(k+n-1)$ " as the golden formula for cycles. Max possible speedup is always just k — if an option says speedup $> k$, eliminate it instantly.

PYQ STYLE:

5-stage pipeline, clock = 2 ns, 100 instructions, no stalls. $T = (5+100-1) \times 2 = 208$ ns; Speedup = $100 \times 5 / 104 \approx 4.8$.

7. Instruction Format & Addressing Modes

QUANTITY	FORMULA
Max Opcodes (fixed-length, n-bit opcode)	2^n distinct opcodes possible
Operand field bits	Instruction Length - Opcode bits = Address bits
Registers addressable	$2^{(\text{address field bits})}$
Effective Address — Indexed	EA = Base/Index Register + Offset
Effective Address — Indirect	EA = Memory[Address field] (one extra memory access)

COMMON PATTERN:

Given total instruction bits and opcode bits, find max number of operands/registers addressable. Or trace a value through register/immediate/direct/indirect/indexed addressing.

SHORTCUT:

Indirect addressing always costs +1 extra memory reference vs Direct addressing. Immediate addressing = 0 extra memory references (value is in the instruction itself).

8. I/O Organization — DMA & Interrupts

QUANTITY	FORMULA
DMA Transfer Time	$\text{Time} = N \times \text{Cycle Time (N = words to transfer)}$
Memory Bandwidth	$\text{BW} = (\text{Bus Width in bits} / 8) \times \text{Clock Frequency [Bytes/sec]}$
CPU slowdown due to DMA (cycle stealing)	$\% \text{ slowdown} = (\text{DMA cycles used} / \text{Total cycles}) \times 100$
Interrupt-driven I/O — CPU time per request	$\text{Time} = \text{Interrupt overhead} + \text{ISR execution time}$
Max I/O rate (interrupt mode)	$\text{Rate} = 1 / (\text{Overhead} + \text{ISR time})$

COMMON PATTERN:

Compare % of CPU time consumed by polling vs interrupt vs DMA for a given device transfer rate.

SHORTCUT:

Polling wastes the most CPU time (busy-wait), Interrupt wastes CPU only when servicing, DMA wastes the least CPU (only "steals" individual cycles). Rank: DMA < Interrupt < Polling for CPU overhead.

PYQ STYLE:

Disk transfers 1 word every 4 μs via DMA, CPU cycle = 0.5 μs → 1 cycle stolen every 8 CPU cycles → CPU slowdown $\approx$ 12.5%.